

PROJECT REPORT

Gesture-Based Mouse Controller Using Computer Vision

Technology: Python, OpenCV, MediaPipe, PyAutoGUI

Input: Webcam

Application: Touch-free mouse control

1. Abstract

This project creates a basic touch-free mouse controller using a webcam. OpenCV captures live video, MediaPipe detects hand landmarks, and PyAutoGUI converts the detected hand position into operating-system mouse actions. The index fingertip controls cursor movement, while a small distance between the thumb and index fingertip is interpreted as a click.

2. Objectives

- Capture live webcam video.
- Detect one hand in real time.
- Track the index and thumb fingertips.
- Map hand position to screen coordinates.
- Smooth cursor movement.
- Recognize a pinch gesture as a click.
- Provide live visual feedback and a safe exit key.

3. Technologies Used

Technology	Role
Python	Program implementation
OpenCV	Video capture, conversion, display and drawing
MediaPipe Hands	21-point hand landmark tracking
PyAutoGUI	Mouse control
math	Euclidean distance calculation

4. Working Principle

- Step 1:** Open the default webcam with OpenCV.
- Step 2:** Flip each frame horizontally for mirror-like interaction.
- Step 3:** Convert BGR video to RGB for MediaPipe.
- Step 4:** Detect hand landmarks with MediaPipe Hands.
- Step 5:** Read landmark 8 (index tip) and landmark 4 (thumb tip).
- Step 6:** Convert normalized index coordinates to screen coordinates.
- Step 7:** Apply interpolation-based smoothing before moving the mouse.
- Step 8:** Calculate the distance between the two fingertips.
- Step 9:** If distance is below 30 pixels, perform a mouse click.
- Step 10:** Show the processed camera frame and exit when q is pressed.

5. Key Code Logic

Cursor mapping and smoothing:

```
target_x = int(index_tip.x * screen_width)
target_y = int(index_tip.y * screen_height)

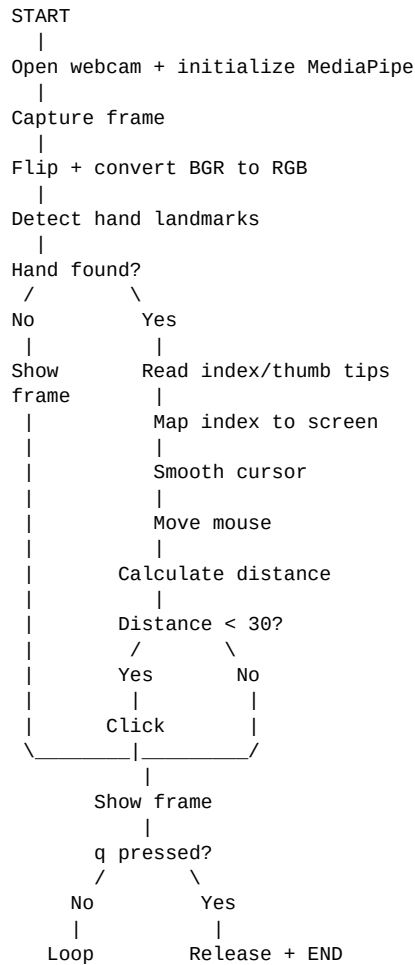
curr_x = prev_x + (target_x - prev_x) / smoothing
curr_y = prev_y + (target_y - prev_y) / smoothing
```

Click recognition:

```
distance = math.hypot(tx - ix, ty - iy)

if distance < 30:
    pyautogui.click()
    pyautogui.delay(0.2)
```

6. Algorithm



7. Advantages

- No physical mouse is required.
- Uses an ordinary webcam.
- Demonstrates real-time computer vision.
- Simple gesture model is easy to understand.
- Can be extended with more gestures.

8. Limitations

- Tracking depends on lighting and camera quality.
- Basic screen mapping is not calibrated.
- Only one hand is tracked.
- A fixed 30-pixel threshold may vary with camera resolution.
- The supplied click logic can generate repeated clicks while the pinch is held.
- FAILSAFE is disabled in the supplied program.

9. Future Enhancements

- Add click debouncing/state detection.
- Add right-click and double-click gestures.
- Add scrolling gestures.

- Add camera-to-screen calibration.
- Add configurable sensitivity and smoothing.
- Support two-hand commands.
- Add an on-screen gesture/status indicator.

10. Testing

Test	Expected Result
Webcam available	Live camera window opens
Hand visible	Landmarks are drawn
Index moves	Cursor follows hand
Thumb near index	Click is triggered
No hand	Cursor remains unchanged
Press q	Program exits and resources are released

11. Conclusion

The project demonstrates how hand-tracking computer vision can provide a practical touch-free mouse interface. Combining MediaPipe landmarks, OpenCV video processing and PyAutoGUI automation allows ordinary hand movements to be translated into mouse actions. The implementation is suitable as an educational foundation for more advanced gesture-controlled applications.

12. References

OpenCV documentation; MediaPipe Hands documentation; PyAutoGUI documentation; Python math module documentation.